

```
"""
```

UMA Semantic Constants.

NOTE ON THE SILVER RATIO:

Earlier versions claimed $E_{DC}/kT = 3 - 2\sqrt{2} = 1/\delta_S^2$ as a fundamental constant of the GENERIC Lagrangian. This was wrong.

Across 50 independent trajectories, E_{DC}/kT ranges from 0.08 to 1.93 with mean 0.77 and std 0.65. The value 0.1713 matched Silver to 0.16% only for seed=42. It was a coincidence, not a derived constant.

The correct closure thresholds are in `calibrate.py` and `friction.py`.

WHAT REMAINS VALID:

- The Engine3 complex state $z = x((1-\sqrt{2}) + i\sqrt{e})$ is a well-defined encoding. It maps text to a complex number via a specific algebraic structure.
- The Silver angle $\arctan(\sqrt{e}/(1-\sqrt{2})) = 104.1$ degrees is the $\arg(z_{\text{seed}})$ – a real property of the encoding.
- These are useful as a DESIGN CHOICE for the encoding, not physics.

```
"""
```

```
import math
```

```
SQRT2 = math.sqrt(2)
```

```
SQRT_E = math.sqrt(math.e)
```

```
# Silver Ratio – valid as a mathematical object, NOT as a physics constant
```

```
DELTA_S = 1.0 + SQRT2
```

```
DELTA_S_SQ = DELTA_S**2
```

```
INV_DELTA_S_SQ = 1.0 / DELTA_S_SQ #  $3 - 2\sqrt{2} \approx 0.17157$ 
```

```
C1_SILVER = 1.0 - SQRT2 #  $\approx -0.41421$ 
```

```
C2_SILVER = 2.0 - SQRT2 #  $\approx 0.58579$ 
```

```
SILVER_ANGLE = math.degrees(math.atan2(SQRT_E, C1_SILVER)) #  $\approx 104.1$  deg
```

```
# Engine3 complex state seed
```

```
#  $z = x * ((1-\sqrt{2}) + i\sqrt{e})$ 
```

```
# This is a DESIGN CHOICE for the text encoding, not derived from physics.
```

```
Z_SEED_REAL = C1_SILVER
```

```
Z_SEED_IMAG = SQRT_E
```

```
def engine3_complex_state(x: float) -> complex:
```

```

    """Map binary weight x to Engine3 complex state."""
    return complex(x * Z_SEED_REAL, x * Z_SEED_IMAG)

def engine3_E(x: float) -> float:
    """E = |z_c|^2/2 for the Engine3 complex state."""
    z = engine3_complex_state(x)
    return (z.real**2 + z.imag**2) / 2.0

if __name__ == "__main__":
    print("UMA Semantic Constants")
    print(f"  Silver Ratio delta_S = {DELTA_S:.6f}  (mathematical, not physics)")
    print(f"  1/delta_S^2              = {INV_DELTA_S_SQ:.6f}")
    print(f"  Silver angle              = {SILVER_ANGLE:.2f} degrees")
    print(f"  sqrt(e)                   = {SQRT_E:.6f}")
    print()
    print("  NOTE: These are encoding design constants, not GENERIC equilibrium v
    print("  Use calibrate.calibrate_closure(cfg) for physics-grounded thresholds

# -----
# Backward-compatible stubs (used in engine.py, venturi – kept for now)
# These return calibrated values, NOT Silver Ratio claims
# -----

CALIBRATED_E_TARGET      = 3.837e-3
CALIBRATED_DZ_DT_FLOOR  = 2.253e-3
CALIBRATED_FIELD_SCALE  = 1.752e-1

def silver_E_target(kT: float) -> float:
    """Returns calibrated E_target. Name kept for backward compat."""
    return CALIBRATED_E_TARGET

def silver_dz_dt_floor(kT: float, lam: float, N: int, dt: float) -> float:
    """Returns calibrated dz_dt floor. Name kept for backward compat."""
    return CALIBRATED_DZ_DT_FLOOR

def silver_field_scale(kT: float, N: int = 32, rho_star: float = 1.0) -> float:
    """Returns calibrated field scale. Name kept for backward compat."""
    return CALIBRATED_FIELD_SCALE

# -----
# H[psi]-based calibrated thresholds (from 50 GENERIC trajectories)

```

```
# These track the ACTUAL Hamiltonian, not the  $|z|^2/2$  proxy
# -----

CALIBRATED_H_TARGET      = 1.092794e+00    # mean(H[psi])
CALIBRATED_H_STD         = 2.550512e-01    # std(H[psi])
CALIBRATED_DH_DT_FLOOR  = 1.292312e+00    # mean(|dH/dt|) - thermal noise rate
CALIBRATED_DH_DT_P25    = 5.191530e-01    # lower quartile (tighter threshold)
```